

Process mining-driven automated generation of threat intelligence artifacts: from processes to Attack Flow models

Marcelo Rodríguez¹, Gustavo Betarte¹, and Daniel Calegari²

¹ Instituto de Computación, Facultad de Ingeniería, Universidad de la República
{marcelor,gustun}@fing.edu.uy

² Universidad ORT Uruguay calegari@ort.edu.uy

Abstract. Process Mining (PM) has become a valuable technique for uncovering behavioral patterns in attack traces extracted from cybersecurity system logs. It moves beyond isolated event analysis to provide a comprehensive view of attack processes. However, the notation used in PM to represent this knowledge is not the most common in cybersecurity. Attack Flow is a domain-specific language designed to formalize how attackers combine and sequence tactics, techniques, and procedures to achieve their objectives. It builds on the detailed taxonomy of the MITRE ATT&CK framework to better represent, analyze, and share adversarial behaviors. In this paper, we propose a method for generating Attack Flow models from PM by linking elements in process models with concepts in the Attack Flow language. To demonstrate and evaluate this mapping, we analyze models created from behavioral analysis of the ransomware WannaCry using PM techniques. Our results indicate that this approach can simplify threat representation and enable more scalable, data-driven cybersecurity analysis.

Keywords: Process Mining · cyber-security · attacker behavior · Attack Flow

1 Introduction

Interviews with industry and government organizations reveal a strong need for security experts to create threat models [4]. A structured description of cyber-adversary behavior is fundamental to analyzing, detecting, and responding to security incidents [6]. Attack Flow [11] is a formal language for modeling how attackers combine and sequence offensive techniques to achieve their goals. It relies on the MITRE ATT&CK framework [16], which provides a detailed taxonomy of tactics, techniques, and procedures (TTPs) used by malicious actors.

The primary objective of Attack Flow is to enable cybersecurity analysts to visualize, observe, and share patterns of adversarial behavior. This broader view extends beyond individual events, facilitating the analysis of complete sequences of actions. The Attack Flow-oriented perspective is beneficial for understanding an attacker's operational logic and anticipating potential compromise vectors. Automated assistance in Attack Flow generation offers a key benefit by enabling

professionals to accelerate their work. However, current solutions demand considerable manual labor and specialized expertise to translate system knowledge.

Process Mining (PM) techniques [1] are used to comprehend the intricate behavior of attackers. In previous work [13,14], we presented an approach that combines PM with MITRE ATT&CK to discover process models of attack strategies. Our approach involves mapping low-level system events to corresponding event labels from the MITRE ATT&CK taxonomy, increasing the abstraction level for attacker profiling. Experiments with real cyberattack data support the validity of the proposed approach. A particular focus is placed on logs of attacks carried out by human actors (utilizing the PWNJUTSU dataset [3]) and a case study based on the automated behavior of the WannaCry ransomware [10].

Nevertheless, although behavioral models mirror taxonomies and nomenclature pertinent to the cybersecurity field, the language employed is not the most prevalent among analysts. We consider linking the process models derived from identifying attacker behavior with Attack Flow, potentially via an automated translation mechanism, thereby fostering a more intuitive integration of PM as an analytical tool within the cybersecurity discipline.

In this paper, we propose a method for generating Attack Flow models from process models, based on a mapping definition that considers control-flow patterns [15]. As a first assessment of the proposal, we use models generated from the behavioral analysis of WannaCry.

The rest of the paper is structured as follows. In Section 2, we present background information on the use of PM to model cyberattacker behavior. Then, in Section 3, we propose the systematic transformation from process models to Attack Flow models, and in Section 4, we present an exploratory example. In Section 5, we discuss the primary results and the challenges they present. Finally, in Section 6 we conclude and present ideas for future work.

2 A PM-based method for attacker profiling

In [13,14], we explored attacker profile discovery using PM and the MITRE ATT&CK taxonomy to provide a structured, model-based understanding of strategies. We defined a four-phase method, as shown in Figure 1, which classifies attacker actions from system logs and is compatible with standard PM methods. The four phases are **Enactment**, where attack strategies are executed; **Extraction**, where data creates event logs labeled with the MITRE ATT&CK taxonomy; **Discovery**, discovering process models of attacker behavior; and **Analysis**, where experts analyze results.

The MITRE ATT&CK framework categorizes activities, serving as a cybersecurity standard for threat detection and mitigation. It details tactics, techniques, and procedures (TTPs) used by malicious actors. Tactics are strategic goals, techniques are methods to achieve them, and procedures are real-world implementations. Our method decomposes the attack into its components and identifies their relationships, enabling a clearer understanding of the attack sequence. For example, Figure 2 shows a Petri net of WannaCry ransomware be-

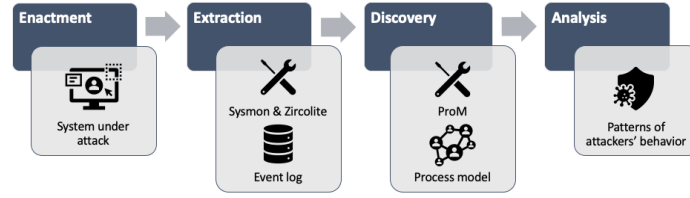


Fig. 1: A method for attack discovery (from [14])

havior, with transitions representing actions or events aligned with the MITRE ATT&CK framework. The labelled transitions represent tactics, but techniques can also be used for finer-grained labelling.

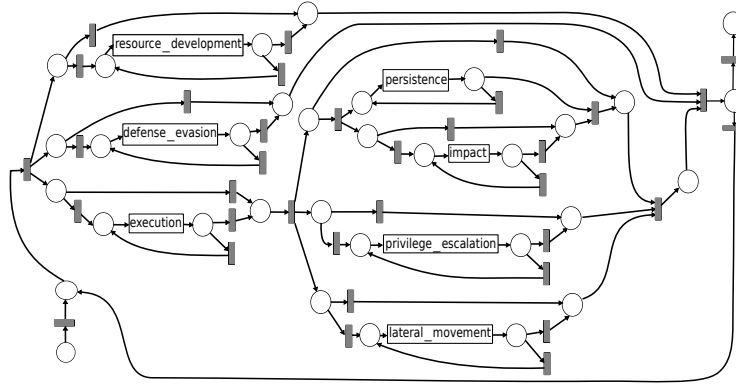


Fig. 2: A discovered Petri Net of the WannaCry ransomware

2.1 The Attack Flow language

STIX (Structured Threat Information Expression) [12] is a language for structured cyber threat information exchange, based on the Attack Flow data model. It represents cyber threat intelligence (CTI) in a human- and machine-readable form. From a formal view, Attack Flow extends STIX by modeling an attacker’s behavior in campaigns. Its key concepts, added by Attack Flow [11] are shown in Table 1. An Attack Flow model details a sequence of attack steps, referencing STIX objects. Its key element is the *action*, representing attacker behavior. While often based on MITRE ATT&CK techniques, *actions* can be customized for specific analyses or environments. *Actions* may depend on *assets*, which describe impact or resources, aligned with STIX objects for consistency and reusability within threat intelligence. *Operators* (AND/OR) combine attack paths with Boolean logic—OR requires one path, AND all paths. Although paths can also

connect directly to the same action, this method may cause ambiguity about execution logic. *Conditions* show outcomes or states from actions, allowing flow splits based on success or failure, clarifying complex relationships, especially when paths are disconnected.

Table 1: Elements of the Attack Flow language

Element	Behavior
Attack Flow	Represents the general flow of the attack.
Attack Action	Represents a specific action within an attacker’s behavior.
Attack Asset	Denotes any object that is the target or affected by an action.
Attack Operator	Combines multiple attack paths using Boolean logic.
Attack Condition	Represents a state or outcome resulting from an action.

3 From PM processes to Attack Flow models

Several modeling formalisms have been adopted to visualize and analyze process models, each with specific characteristics concerning expressiveness, readability, and analytical capabilities. Control-flow patterns [15] provide an abstract description of the control-flow perspective. We use them to express how process models that depict attacker behavior can be translated into Attack Flow models.

As part of the PM-based method described in Section 2, we have used the *Inductive Miner* algorithm [8] due to the favorable results obtained in the context of cybersecurity [9]. The algorithm discovers Process Trees models, an abstract representation of a block-structured workflow net. For this reason, we have considered the basic control patterns supported by the Inductive Miner, as described in Table 2, which are generally supported by most discovery algorithms and general enough to represent a large number of scenarios [2].

We have built a prototype of the automated generation of Attack Flow models from process trees using the PM4Py and Stix2 Python libraries. On the site ³, we share the corresponding Python code and provide execution examples, along with experimental results on attacker models.

Activities: Beyond the specific behavioral patterns, the most critical elements identified in the models are the milestones that shape the attacker’s activity. In our research, it was proposed that the tactics defined by the MITRE ATT&CK taxonomy represent the basic activities carried out by the attacker. These activities are modeled as transitions. In the current context, a direct link is established between these activities and the actions shown in Attack Flow. This allows a consistent integration between the models gained through PM and their formalization within the Attack Flow. *Condition objects* allow flows to be bifurcated

³ <https://gitlab.fing.edu.uy/gsi/attacker-profiling/pm2attackflow>

Table 2: Considered control-flow patterns

Class of pattern	Control-Flow
Basic Control-Flow:	Sequence Parallel Split Synchronization Exclusive Choice Simple Merge
Advanced Branching + Sync:	Multi-Choice Structured Sync. Merge
Iteration:	Structured Loop
Termination:	Implicit Termination

according to the result (success or failure) or provide a description of an action’s execution without implying a decision (i.e., adding context). In our context, as a general rule, they will not only express logical conditions but will also help contextualize actions by preceding or following each activity.

Sequence: This pattern describes a sequential dependency relationship, where a task within a process can only proceed after the completion of a preceding task. In Attack Flow, sequences of actions performed by a cyberattacker are shown through directed connections between them, with arrows indicating the transition from one action to another. The figure 3(a) demonstrates this pattern using a Petri net and the corresponding Attack flow representation.

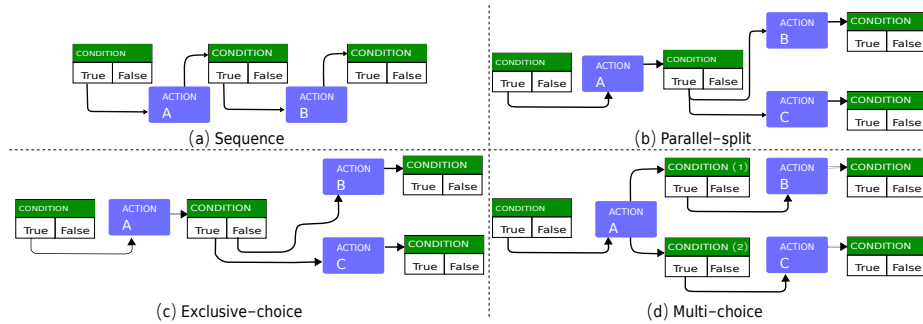


Fig. 3: Sequence and split patterns in Attack Flow

Parallel Split: The divergence of a branch into two or more parallel branches is expressed by this pattern. After the divergence, each branch executes concurrently. In Attack Flow (see Figure 3(b)), a cyber attacker may utilize a variety of techniques that can be executed concurrently within a given scenario. From an initial action, multiple arrows can originate, leading to parallel attack paths.

Synchronization: This pattern is employed to model the convergence of two or more execution branches into a single subsequent branch, contingent upon the successful completion of all preceding branches. In the context of Attack Flow, this logic is denoted by the AND operator, indicating that the execution of all incoming attack paths must be completed for the flow to proceed. Figure 4(a) presents a visual representation of the pattern above. Again, it is essential to emphasize that the conditions preceding the action are not strictly necessary.

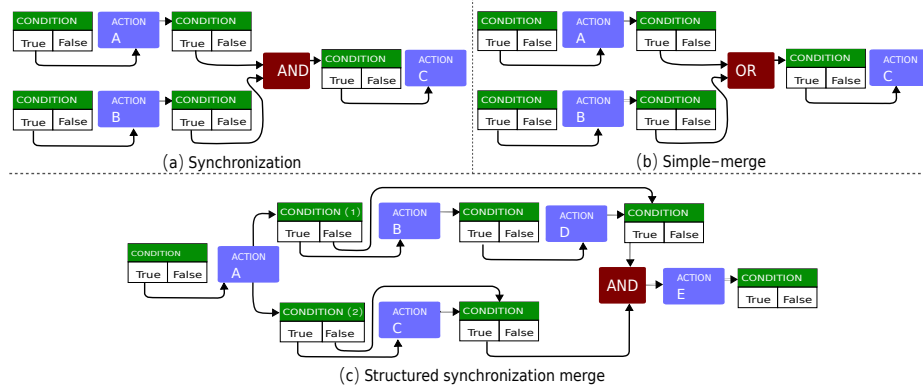


Fig. 4: Synchronization patterns in Attack Flow

Exclusive Choice: This pattern illustrates the branching of a single path into two or more alternative paths, where control is transferred to only one of them, as determined by a decision mechanism. In Attack Flow, this type of behavior can be modeled using conditions. Each condition delineates two potential branches: a true branch, which is triggered when the underlying condition is met, and a false branch, which is followed otherwise. In the context of attack modeling, this structure helps represent how an attacker reacts to the success or failure of a given action (see Figure 3(c)).

Simple Merge: This pattern describes the merging of two or more branches into a single subsequent branch (see Figure 4(b)). The activation of only one incoming branch is sufficient to enable the flow to continue to the next branch. However, a challenge associated with this approach is that it is not always possible to guarantee that the branches converge *asynchronously*, i.e., that they are not activated simultaneously. In Attack Flow, there is no specific *operator* that represents *exclusive OR* convergence. However, the models obtained through PM thus far exhibit a systematic pattern, preceded by *exclusive choice pattern*. In such cases, asynchronous behavior can be guaranteed by using the OR operator (preceded by the *condition object* for the branch), which aligns with the expected semantics of the pattern.

Multi-Choice: This pattern enables the execution flow to be selectively split into multiple concurrent threads, based on a decision mechanism that can activate one or more of the outgoing branches. In Attack Flow, this behavior can be modeled using specific conditions associated with each branch. These conditions independently determine whether a given thread should be activated. Figure 3(d) illustrates this behavior graphically, based on the use of two conditions.

Structured Synchronizing Merge: This pattern represents the convergence of multiple branches that a Multi-Choice previously forked. For the subsequent branch to be enabled, all incoming branches activated during execution must be available. In other words, continuation of the flow requires synchronization of all previously selected paths. Figure 4(c) illustrates this pattern schematically.

Structured Loop: This pattern (see Figure 5) enables the repeated execution of a process or sub-process by incorporating a control condition that can be evaluated before (pre-test) or after (post-test) each iteration, thus determining the continuity of the loop. The pattern’s structure is defined by a single entry and exit point, ensuring well-defined control semantics. It should be noted that, although this type of pattern is not explicitly prohibited in Attack Flow, its use is infrequent in practice.

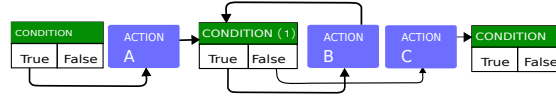


Fig. 5: Structured loop pattern in Attack Flow

Implicit Termination: The conclusion of a process instance is marked by the absence of pending work items and the absence of a deadlock situation. To this end, it is imperative to establish an objective criterion that can ascertain the successful termination of the process. In the context of Attack Flow, this condition may be expressed as the utilization of a *condition object* to delineate the termination criterion for malicious activity.

4 Discovering the Attack Flow model of WannaCry

WannaCry is a ransomware first identified in a worldwide attack in May 2017. The incident’s effects lasted for a long time, causing substantial financial losses and highlighting the vulnerability to ransomware attacks. Consequently, it has been the subject of numerous studies, e.g., [5]. Table 3 shows WannaCry’s attack tactics and techniques mapped to the MITRE ATT&CK framework. Each row lists a tactic and technique, with a code and brief description of malware’s use in its attack chain. WannaCry performs activities like **persistence** via malicious

services; **discovery** by searching files, configs, and systems; **lateral movement** by exploiting vulnerabilities; **evading defenses** by hiding files and changing permissions; **impact** by encrypting data, stopping services, and removing recovery points; and **command and control** via encrypted channels.

Table 3: Tactics and techniques associated with WannaCry [10]

Tactic	Technique	Use
Persistence	T1543	Creates the service <code>mssecsvc2.0</code>
Impact	T1486	Encrypts user files and demands that a ransom
Command and Control	T1573	Uses Tor for command and control traffic
Lateral Movement	T1210	Exploits SMBv1 to spread to remote systems
Discovery	T1083	Searches several user files by extension before encrypting
Defense Evasion	T1222	Uses <code>attrib</code> and <code>icacls</code> to hide files and grant full access
Impact	T1490	Disables OS recovery using system commands
Lateral Movement	T1570	Attempts to copy itself to remote computers
Discovery	T1120	Scans for newly attached drives
Lateral Movement	T1563	Executes malware via remote session enumeration
Discovery	T1018	Scans its local network segment for remote systems
Impact	T1489	Attempts to kill services to encrypt their data stores
Discovery	T1016	Attempt to identify its local network segment
Execution	T1047	Uses <code>wmic</code> to delete shadow copies

As described in [14], we registered five distinct instances of WannaCry execution and used the Inductive Miner (IM) algorithms. The model includes seven distinct activities carried out by the ransomware. Figure 2 showed a Petri Net of WannaCry ransomware behavior, discovered via the PM-based attacker profiling method in the same section. Although it shows some discrepancies regarding the tactics listed in Table 3, its main contribution is in illustrating the sequence of actions described by these tactics. It demonstrates expected behavior: first disabling defenses, then executing malicious tactics like persistence, privilege escalation, lateral movement, and impact. These activities are part of a continuous cycle, as ransomware runs uninterrupted, repeating actions such as searching for new files to encrypt and connecting to its command and control (C&C). In contrast to conventional business processes, which have a defined start and end, this behavior reflects the inherent nature of ransomware.

Without loss of generality, the present analysis and transformation correspond to one of these tactics: **execution**. For the Petri Net **execution sub-model** (Figure 6a), a sequence of places and transitions is presented that can be derived from two main paths. The first route continues directly to later stages of the attack. The second route activates the transition labeled *execution*. This transition comprises a series of techniques that use code controlled by an adversary on a local or remote system. Upon the attainment of this objective, the flow facilitates progression to subsequent phases of the attack, including: *impact*, *lateral movement*, *privilege escalation*, or *persistence* (see Figure 2). Furthermore, the model displays feedback loops, which can be interpreted as representing persistent behavior on the part of the attacker or as repetitive attempts in response to partial failures during the execution of the attack.

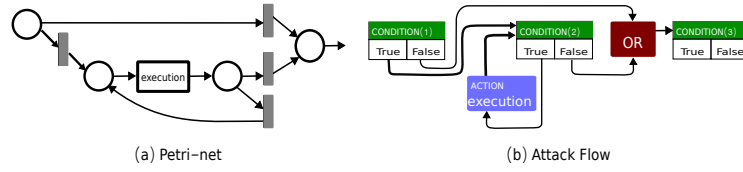
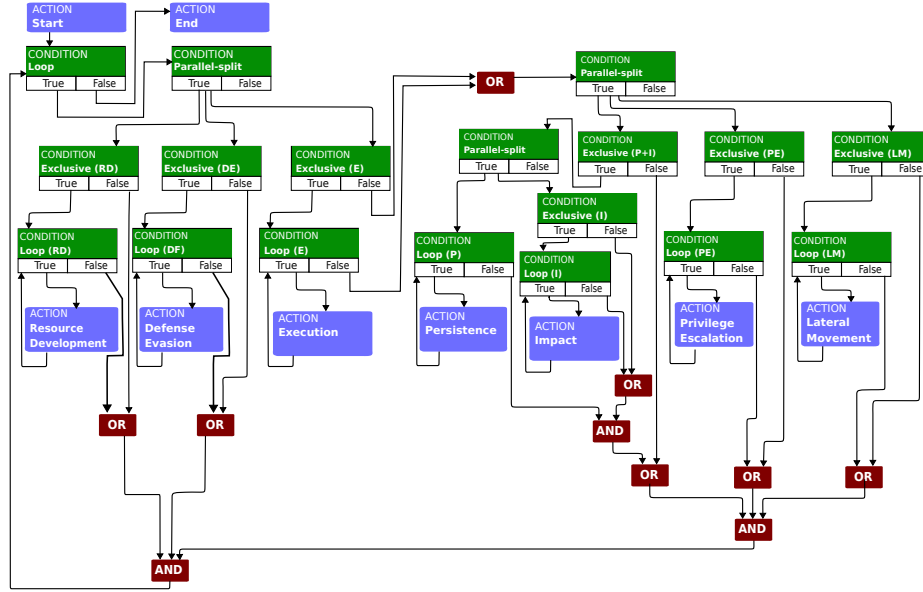
Fig. 6: *execution* tactic sub-model [14]

Fig. 7: Attack Flow model for Wannacry

Figure 6b shows the Attack Flow model derived by using the mapping proposed in Section 3. Figure 7 presents the simplified Attack Flow model for Wannacry (adapted from ⁴), which is derived from the Petri Net shown in Figure 2. To model the behavior using Attack Flow logic, the model begins with the “*Infected Machine*” condition, representing the initial system compromise. From there, the flow branches into multiple paths, composed of preconditions and post-conditions, that determine the execution and validity of intermediate states along the attack chain. Tactical actions represent specific activities executed by the attacker. They are aligned with the MITRE ATT&CK framework’s tactics, such as *execution*, *defense evasion*, *privilege escalation*, *persistence*, *lateral movement*, and *impact*. Logical connectors (AND/OR) allow the modeling of conditional relationships between different flow branches: OR connectors indicate possible alternative routes. In contrast, AND connectors express the need for multiple conditions to be fulfilled simultaneously, allowing the attack to progress.

⁴ <https://gitlab.fing.edu.uy/gsi/attacker-profiling/pm2attackflow/>

5 Discussion and related work

Our work demonstrates that attacker behavioral models derived from PM can be converted into structured attack flows, enabling cybersecurity professionals to create baseline models from event logs. Current research can be categorized into two main approaches: utilizing logs from compromised systems to develop models in standard languages, and applying PM techniques for the automatic creation of models. However, combining both methods is still limited.

The latest Attack Flow version (3.0.0) now includes a usage guide focused on a defensive posture⁵ with a manual example using Windows event logs like Sysmon and security events to show attack patterns. In [4], the authors propose an automated framework for modeling attack paths and encoding this information with Attack Flow, highlighting the challenges of manual effort and expertise required for accurate threat modeling tools.

To develop understandable and user-friendly models within the cybersecurity community, we highlight the work of [7], which presents a PM-based method for creating attack trees. Our main difference is the use of the MITRE ATT&CK taxonomy to represent attacker behavior, allowing easy integration with cybersecurity tools aligned with this taxonomy and enabling conversion into the Attack Flow language. Automating the initial Attack Flow generation from system event logs helps minimize biases or errors introduced by less experienced human analysts. Nonetheless, current PM models focus only on control flow, neglecting data aspects. This is evident in the handling of *preconditions* and *postconditions*. While these are syntactically recognized as necessary for evaluating action feasibility and state validity, automatically deriving precise logical expressions remains unfeasible at this stage. Currently, cybersecurity analysts must manually provide this information. Although human-in-the-loop is generally desirable, it is necessary to study techniques that mitigate the need to include all pre- and post-conditions manually. To this end, we could consider exploring object-centric process mining to analyze the behavior of the objects involved (Attack Asset) and at least partially derive the observed conditions.

Another important aspect related to the data perspective is the integration of the *asset objects* from the Attack Flow schema, along with any additional STIX objects. These are useful for illustrating the impact of the attacker’s actions on elements of the compromised environment. Generally, asset objects provide informative insights, describing which assets are affected by specific actions or which environmental resources are used in later techniques within the same attack flow. To automatically include this information in the model, a detailed analysis of the attack execution data is required. This analysis helps identify the connections between observed events and the involved assets. However, this task faces several challenges, particularly in the areas of information extraction, semantic correlation, and contextualizing the data within system logs.

⁵ https://center-for-threat-informed-defense.github.io/attack-flow/usage_guides/defensive-posture/

An interesting aspect of this discussion is the observation of loop-related patterns. In the standard Attack Flow examples examined [11], these patterns are rarely observed. However, in our research, they have appeared multiple times. We believe this frequency may partly stem from the modeling approach, which highlights tactics rather than specific techniques. Furthermore, their occurrence reflects typical attacker behavior, especially when considering the exploration of multiple options to achieve a target. This pattern is particularly relevant for human attackers, whose decision-making often involves trial-and-error cycles influenced by environmental factors and system responses.

A consistent obstacle in our research has been the scarcity of suitable datasets. The WannaCry case was chosen since it is documented in the MITRE ATT&CK framework [10], discussed in the literature [5], and can be safely recreated in controlled environments, allowing us to collect relevant system logs. The main challenge remains generating usable system logs for process mining.

6 Conclusion and Future Work

We proposed a method for creating Attack Flow models from process models that represent malicious activities identified through process mining of system logs. Although the data is not exhaustive, security experts can refine the models by adding domain-specific knowledge. We established a translation process based on workflow patterns to convert process models into Attack Flow models. This could enhance tools to include cybersecurity analysis, with ongoing refinement, until a reference model is established, which can then be imported into Attack Flow Builder, a free, open-source app for attack flows, for further refinement. Alternatively, the translation could be integrated directly into Attack Flow Builder for importing discovered Petri nets.

This type of tool could significantly impact how Attack Flow models are created. This is because it can work directly with data from real incidents, like system logs. While further research is needed to develop more valuable and practical models, this method not only supports existing modeling techniques but also enhances teamwork between process analysts and security experts, fostering a more unified, automated, and practical understanding of attacks.

Concerning future research, we can consider incorporating advanced patterns that capture specific types of behavior. Similarly, analysis of attacker behavior models has revealed patterns indicative of explicit decision-making processes. Common patterns, such as Exclusive-Choice and Multi-Choice, are often identified in these models. These patterns depict scenarios where control flow diverges toward one among multiple options. In Petri nets or their corresponding attack flow representations, such decisions are clearly expressed in the model structure. However, there is no information about the criteria for choosing one path over another, nor is there a formal method for representing these decisions using logical predicates. Decision mining [1] is proposed as a strategy to infer logical rules from data. We believe incorporating this method is especially important for at-

tack analysis, as it can help uncover how specific attributes or conditions in a process influence the choice of different attack pathways.

Furthermore, abstract process patterns allow us to extend the approach beyond a particular algorithm. Further work is also needed for developing deeper quantitative and qualitative evaluations for validating the correctness of the transformation, the usefulness of the generated models, and addressing the generalizability of the approach.

References

1. van der Aalst, W.M.P.: Process Mining - Data Science in Action, Second Edition. Springer (2016)
2. Andree, K., et al.: Discovery of workflow patterns - A comparison of process discovery algorithms. In: 29th CoopIS, Proceedings. LNCS, vol. 14353, pp. 257–274. Springer (2023)
3. Berady, A., et al.: Pwnjutsu: A dataset and a semantics-driven approach to retrace attack campaigns. *IEEE Transactions on Network and Service Management* **19**(4), 5252–5264 (2022)
4. Cheh, C., et al.: Lowering the barrier: An automatic attack flow generation framework for non-security experts. In: Adi, K., et al. (eds.) *Foundations and Practice of Security*. pp. 77–93. Springer Nature Switzerland, Cham (2025)
5. Chen, Q., Bridges, R.A.: Automated behavioral analysis of malware: A case study of wannacry ransomware. In: 16th IEEE ICMLA. pp. 454–460 (2017)
6. Dehghantanha, A., Conti, M., Dargahi, T.: *Cyber Threat Intelligence*. Springer, Cham (2018)
7. Konsta, A.M., et al.: Attack tree generation via process mining. In: *Leveraging Applications of Formal Methods, Verification and Validation. REoCAS Colloquium in Honor of R. De Nicola*. pp. 356–372. Springer Nature Switzerland, Cham (2025)
8. Leemans, S.J.J., Fahland, D., van der Aalst, W.M.P.: Discovering block-structured process models from incomplete event logs. In: *Application and Theory of Petri Nets and Concurrency*. pp. 91–110. Springer (2014)
9. Macak, M., Daubner, L., Fani Sani, M., Buhnova, B.: Process mining usage in cybersecurity and software reliability analysis: A systematic literature review. *Array* **13**, 100120 (2022)
10. MITRE: ATT&CK: WannaCry-associated techniques. <https://attack.mitre.org/software/S0366/> (2024)
11. MITRE: Attack Flow. <https://center-for-threat-informed-defense.github.io/attack-flow/> (2025)
12. OASIS: Structured Threat Information eXpression (STIX). <https://oasis-open.github.io/cti-documentation/stix/intro> (2023)
13. Rodríguez, M., Betarte, G., Calejari, D.: Discovering attacker profiles using process mining and the MITRE ATT&CK taxonomy. In: 12th Latin-American Symposium on Dependable and Secure Computing, LADC 2023. pp. 146–155. ACM (2023)
14. Rodríguez, M., Betarte, G., Calejari, D.: A process mining-based method for attacker profiling using the MITRE ATT&CK taxonomy. *J. Internet Serv. Appl.* **15**(1), 212–232 (2024)
15. Russell, N., van der Aalst, W.M.P., ter Hofstede, A.H.M.: *Workflow Patterns: The Definitive Guide*. MIT Press (2016)
16. Strom, B., Applebaum, A., Miller, D., Nickels, K., Pennington, A., Thomas, C.: MITRE ATT&CK: Design and philosophy. Tech. rep., The MITRE Corp. (2018)